

Mirza Bot: PHP → Python rewrite parity audit

An independent, static-only review of the `rewrite/python` branch against production `main`, treating every behaviour in `main` as a requirement.

BASELINE

63.6k LOC

REWRITE LOGIC

8.0k LOC

FEATURES SCORED

96

WEIGHTED PARITY

≈ 55%

CRITICAL DEFECTS

4

DATE

2026-08-25

01 Executive summary

The rewrite is a **well-structured, genuinely more maintainable skeleton that is not yet feature-complete** — and, critically, is broken on the one path the whole product exists for: **taking money and delivering a config**. Architecturally it is a real step up from the PHP monolith: a plugin registry for panels, payments and jobs; an ORM with real foreign keys and indexes; JSONB settings; Fernet-encrypted panel passwords; idempotent payment claiming; and proper Telegram Mini-App signature validation. As a *foundation to build on*, it succeeds.

Measured against the actual instruction — **full feature parity with `main`** — it is roughly half-done, and the missing half includes the revenue path. Three independent defects each break online payments on their own:

- #1 `PaymentService.settle_wallet_topup(...)` is called in four places but **defined nowhere**. Every wallet top-up settlement — Zarinpal, IranPay, Plisio, NowPayments, card-to-card, Telegram Stars — raises `AttributeError` at runtime.
- #2 Paying for a **product** through any online gateway **never provisions it**. The bot checkout creates the order with no `invoice_id`, so settlement treats the purchase as a wallet top-up (which then hits #1).
- #3 The customer is **never notified** when a gateway payment settles (`_notify_user(None, ...)`) and the admin payment report is a **no-op placeholder**.

The single biggest risk is therefore not "some features are missing." It is that **the money path looks finished** — there are gateway modules, a passing test suite, and a self-authored `PARITY.md` claiming  almost everywhere — **yet it does not work**

end-to-end, and the tests are green because they never exercise the broken branch. Anyone trusting the self-reported matrix would ship a bot that accepts payment and hands out nothing.

Beyond payments, the admin surface is a fraction of the original (~25 mostly create-and-list commands versus ~177 branches in `admin.php`; no multi-admin support, no in-bot text customization, no per-panel pricing or inbound editing). Several scheduled jobs are stubs or point at the wrong destination — expiry reminders go to an admin channel instead of the customer, the lottery job is absent, and the gift job is literally `return None`. A number of admin-configurable settings are written but never read (lucky-wheel prizes, gift-code percentages).

55%
WEIGHTED PARITY

✗ Not ready to replace main

Far off as a drop-in replacement; **close** as an architectural foundation. Completion clusters in read/display paths, so functional readiness of the purchase flow is materially lower than 55%. Do not cut over until the §8 must-fix list is cleared and a dynamic pass confirms provisioning against real panels.

02 How this pass was conducted

- Both branches read directly from git worktrees; **no code executed**, no dependencies installed, no PostgreSQL or panels contacted.
- Breadth-first, risk-weighted:** every functional area inventoried and mapped; deep line-level scrutiny concentrated on the money path, scheduled jobs, auth, and the data model.
- The rewrite's own `PARITY.md` was treated as **a claim to verify, not evidence** — several of its  entries are contradicted by the code and are called out inline.
- Where a conclusion is inferred rather than proven by reading, it is labelled as such. The honest depth boundary is stated in §10.

03 Feature parity inventory

 Done

 Improved

 Partial

 Buggy / wrong

 Risky

 Missing

4.1 • Purchase & provisioning				
ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-001	Category browsing	Category → product drill-down		buy.py
F-002	Product listing / selection	Filtered by category / location		buy.py, mini_services
F-003	Location choice at checkout	Per-product location honoured		Bot flow has no location pick; _default_panel grabs first non-/all product
F-004	Automated config creation	Create panel user, return config/sub		panels/service.py::create_user
F-005	Wallet purchase	Deduct balance then provision, atomically		Provisions & commits invoice BEFORE charging; failed charge → free config; double-spend race
F-006	Online-gateway product purchase	Pay via gateway → provision product		No invoice_id on order → mis-routed to (broken) wallet top-up; product code discarded
F-007	Discount at checkout	Percent / fixed / limits / per-product		Only fixed applied; percent ignored; used_count not incremented; expiry & DiscountSell unused
F-008	One-purchase-per-plan guard	one_buy_status blocks re-buy		Enforced in Mini-App only, not bot buy path
F-009	QR code for config	QR image of config / sub		Not found in rewrite
F-010	Sub link / /sub proxy	Proxy subscription content		api/subproxy.py; subvip rewrite
F-011	Protocol / inbound config	Product inbounds + protocol table		inbounds passed through; protocol table absent from model

4.2 · Service management

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-012	View purchased services	List invoices + live panel data		services.py, mini_invoices (ownership-checked)

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-013	Renew / extend service	Add time ($\pm$ volume / reset)		Volume math questionable; service_time semantics inconsistent
F-014	Buy extra volume	Add GB to live config, paid		extra_volume modifies panel; payment linkage thin
F-015	Buy extra time	Add days to live config, paid		extra_time; same payment caveat
F-016	Change location (paid)	Recreate preserving quota, remove old		create_user(data_limit=0, expire=now) → dead config; quota dropped
F-017	Revoke subscription link	Rotate sub token		revoke_sub → adapter
F-018	Config / sub retrieval	Re-send config & link		services / mini-app
F-019	User disable / enable service	User pauses a service		Simplified vs legacy disorder flow
F-020	Cancel / delete service	cancel_service + admin review		Table exists; flow unconfirmed

4.3 · Payments & wallet

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-021	Wallet balance & ledger	Balance + ad-hoc adjustments		Append-only balance_ledger (auditable)
F-022	Wallet top-up via gateway	Pay → credit balance		settle_wallet_topup UNDEFINED (called 4×) → AttributeError
F-023	Card-to-card (manual)	Receipt → admin approve → credit		card2card + croncard + paycheck; incomplete + shares broken settle
F-024	Zarinpal	PG v4 verify		Module exists; settlement blocked by F-006/022
F-025	Aqayepardakht	Code-70 replay-safe		Module exists; settlement caveat
F-026	NowPayments	Crypto webhook + poll		Module + poll; settlement caveat

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-027	Plisio	Crypto poll		Module + poll; settlement caveat
F-028	IranPay	Factor poll		Legacy return-in-while bug fixed; settlement caveat
F-029	CubePay (TRON)	HMAC callback + result page		Signed callback, fee math, HTML card; unit-tested offline
F-030	Telegram Stars	Invoice, pre-checkout, settle		Settlement calls undefined settle_wallet_topup → crash
F-031	Payment idempotency	Avoid double-settlement		claim_paid atomic UPDATE... WHERE Unpaid
F-032	Payment report → channel	Rich report to report group		_report() is a no-op placeholder
F-033	User notification on payment	DM buyer on success		_notify_user(None, ...) — bot hardcoded None
F-034	Pending-payment expiry	Expire stale orders		payment_expire job (fail after 1h)

4.4 · Bot UX & onboarding

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-035	/start, menu, deep-link	Menu + ref_capture		menu.py
F-036	Language switch (fa/en/ru/zh)	Per-user language		i18n + middleware
F-037	Accept-rules gate	Must accept before use		Legacy acceptRules not observed
F-038	Forced channel membership	Multi-channel mandatory join		Single channel_lock only; channels table not enforced; _check_member fails open
F-039	Phone / number verification	iran_number blocks purchase		/api/verify sets status; no enforcement in purchase path

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-040	Ban / block enforcement	Blocked users can't act		AuthMiddleware + mini-app check
F-041	FSM / step persistence	Step + scratch in DB		Process-local dicts (buy/topup/broadcast/sessions); lost on restart; multi-worker races

4.5 · Growth & marketing

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-042	Referral link + count	Show link, count referred		misc.affiliates, menu captures referrer_id
F-043	Referral commission payout	Referrer earns on referred buy		porsant_one_buy / reagent_report / status_commission unused
F-044	Cashback rewards	Cashback on spend		Only hardcoded cashback=0 in _settle
F-045	Discount codes (admin)	Create % / fixed, limits		/giftcode creates inert code (percent=0/price=0)
F-046	Gift codes	Redeemable credit + scheduled drops		Consumed count collapses to ≤1; gift job is return None
F-047	Lottery (scheduled draw)	cronbot/lottery.php periodic draw		No lottery job (main has 17, rewrite ships 16)
F-048	Lucky wheel (daily)	Daily spin, admin prizes		Hardcoded WHEEL_PRIZES; /wheelprizes setting never read

4.6 · Agent / reseller

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-049	Agent grant / revoke / list	Multiple agent levels		Single agent_level='a'
F-050	Agent request approval	Request → admin approve		No user notify / panel grant on approval
F-051	Agent types & pricing (n/n2)	Distinct reseller tiers		Not ported

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-052	Agent expiry	Auto-demote expired agents		expireagent job
F-053	Agent panel provisioning	mirza_agent.php		Adapter present; not deep-verified

4.7 · Support & content

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-054	Support contact	Show support handle		misc.support
F-055	Ticketing via departments	Departmental tickets + tracking		extra.py + Departman; admin add/list only
F-056	FAQ / tutorials browsing	Help entries by OS/category		menu help + help CRUD
F-057	Tutorial media (photo/video)	Media-rich tutorials		/addhelp stores text only

4.8 · Admin management

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-058	Stats dashboard	Rich financial / user stats		Counts + revenue only
F-059	User management	Find / block / balance / limits		No maxbuyagent / changeloclimit / expireset / verify mgmt
F-060	Product CRUD	Full create/edit/delete + fields		addproduct + list; no edit/delete; drops most fields
F-061	Panel CRUD	Full editing incl. pricing/inbound		add + list + toggle; no edit; pricing/inbound/test not editable
F-062	Category CRUD	Add/list/delete/reorder		add/list/delete
F-063	Discount / gift CRUD	Full management		Create/list only
F-064	Channel management	Register + lock multiple		add/list + single lock; multi not enforced

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-065	Department CRUD	Manage departments		add/list only
F-066	Manual sell (/sell)	Admin gifts a real plan		Provisions data_limit=0; ignores product vol/days; arbitrary panel
F-067	Broadcast	Text/media/pin/forward, batched		Text only; one DB session per use (O(N))
F-068	Multiple admins + rule levels	admin table, permission levels		AdminGate = only primary admin contradicts PARITY 
F-069	In-bot text customization	Edit any bot string from bot		No settext/bottext handler — headline main feature
F-070	Username generation methods	Multiple configurable strategies		random / number only
F-071	Card-number management	Add/remove/toggle cards		Model exists; no admin handler
F-072	Report-topic configuration	Map reports → forum topics		topicid model exists; no config UI
F-073	Settings toggles	Dozens of operational flags		Web panel exposes ~5 keys
F-074	Payments report (/payments)	Recent payments list		payments_report
F-075	Lucky-wheel prize config	Configure prizes		Writes wheel_prizes; wheel never reads it

4.9 · Scheduled jobs (cron)

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-076	Expiry reminders → user	DM buyer before expiry + renew btns		Only send_report to admin channel; customer never notified
F-077	Volume warnings → user	DM at ≥80% usage		Posts to report channel, not user

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-078	Disable expired configs	Disable on panel + persist		Disables on panel but never commits status
F-079	Re-activate / renew configs	Re-enable renewed configs		activeconfig (commits)
F-080	On-hold activation	Activate on-hold at first use		Job exists but no handler sets on_hold; day-count math bug
F-081	Trial cleanup	Remove expired trials		configtest
F-082	Node uptime monitor	Alert on down nodes		Adapter built with ENCRYPTED password (not decrypted) → auth fails
F-083	Panel uptime monitor	Alert on unreachable panels		uptime_panel
F-084	Automatic backups	pg_dump / mysqldump / sqlite		backup.py present; not deep-verified
F-085	Daily status report	Rich nightly financial report		New-user + counts only

4.10 · Web / HTTP surfaces

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-086	Web admin panel	Full CRUD admin (~90k bytes PHP)		List-only dashboards + 5-key settings; in-memory sessions
F-087	Mini-App backend API	Signed-request user API		Session-token identity, ownership checks, initData HMAC
F-088	Mini-App static serving	Serve the SPA bundle		miniapp_static.py
F-089	Telegram webhook + secret	Receive updates		Validates secret (improvement) but fails open if unset
F-090	Sub proxy	Serve subscription content		subproxy.py
F-091	White-label child bots	Per-child dispatch (botsaz)		whitelabel.py + model; not deep-verified

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-092	Machine API tokens	hash.txt token auth for /api		auth (constant-time) + session dual-auth

4.11 · Data & infrastructure

ID	FEATURE	MAIN BEHAVIOUR	STATUS	NOTES
F-093	DB schema parity	~30 MySQL tables		29 modelled well; protocol table missing; setting redesigned to KV (no compat view)
F-094	Migrations	Schema versioning		0001_initial is create_all in disguise; dead col_defs loop; no real DDL
F-095	Panel password encryption	(plaintext in main)		Fernet at rest (improvement); key from change-me default = known key
F-096	Installer	Apache/PHP/MySQL/certbot		systemd + Caddy + alembic; different stack, unverified

04 Additions in the rewrite

Genuinely useful and mostly aligned with the "improve where reasonable / design for extension" instruction. Crucially, these are **labelled** in `PARITY.md` rather than silently mixed into parity work — that instruction was followed. The problem is the false  parity claims sitting beside them.

- **Plugin registry** for panels, payments and jobs (`@register_plugin`) with runtime enable/disable — the main extensibility win.
- **Append-only balance ledger** — auditable wallet.
- **Idempotent payment claiming** (`claim_paid`) — correct concurrency guard (undermined in practice by the settle bug).
- **Webhook secret validation** — closes a real main gap (fails open if unset).
- **Fernet encryption of panel passwords at rest** (weak-default-key caveat).
- **JSONB settings, real FKs and indexes** on the invoice/payment hot paths.
- **Proper Telegram `initData` HMAC validation** for the Mini-App — stronger than main.
- **Batched / rate-capped** broadcast and payment polling.

05 Architecture & extensibility

This is the rewrite's strongest dimension, and the instruction to "design for future add-ons" was taken seriously rather than producing a literal transliteration.

Concretely:

- **Clean module boundaries:** `handlers/` (bot), `panels/` (provisioning adapters), `payments/` (gateways), `jobs/` (scheduler), `api/` (HTTP), `models/`, `i18n/`. This is a real separation the PHP monolith (a 10.9k-line `admin.php`, a 6.7k-line `index.php`) never had.
- **A genuine plugin system:** `registry/` plus a `contracts.py` defining `PanelAdapter`, `PanelResult` and `Job` interfaces. Adding a panel or gateway is a new class with a decorator — the correct extension seam, and the clearest evidence this is designed to grow.
- **Config-driven where it counts:** a settings KV store and per-panel columns replace scattered PHP constants; the scheduler is toggleable like the legacy `status_cron`.

The extensibility is undercut in three places where the design reverts to hardcoding:

process-local dictionaries for purchase, top-up, broadcast and web-session state (breaks multi-worker and restarts); **single-admin gating** that ignores the `Admin` / `rule` tables; and **hardcoded values** that shadow admin config (wheel prizes, trial limits). The bones are extensible; several muscles are still nailed in place.

06 Code quality & QA findings

■ Critical — blocks launch

- 1 **Undefined `settle_wallet_topup`** — every gateway top-up and Stars settlement crashes. F-022 · F-030
- 2 **Gateway product purchase never provisions** — missing `invoice_id` mis-routes real purchases to the broken top-up branch. F-006
- 3 **Buyer never notified & admin report is a no-op** on settlement. F-032 · F-033
- 4 **Wallet purchase provisions & commits the invoice before charging** — a failed charge only rolls back an already-committed session → free config; TOCTOU double-spend. F-005

■ High

- 1 **Change-location & manual-sell create zero-volume / expired configs.** F-016 · F-066
- 2 **Expiry & volume reminders go to an admin channel, not the customer** — the renewal engine never reaches buyers. F-076 · F-077
- 3 `disableconfig` **never commits** its status change; re-runs every tick. F-078
- 4 **Multiple-admins unimplemented** despite a  claim; secondary admins are locked out. F-068
- 5 **Migrations are** `create_all` **in disguise** with dead code and no compatibility view for the redesigned settings table. F-094 · F-093
- 6 **Process-local state** breaks restarts and any multi-worker deployment; enables the F-005 race. F-041

■ Medium

- Discount engine ignores percentages, usage limits, per-product codes and expiry. F-007 · F-045
- Lucky-wheel & gift-code admin config written but never read. F-048 · F-075 · F-046
- Lottery job and scheduled gift drops absent / stubbed. F-047
- `uptime_node` uses the encrypted password without decrypting. F-082
- Referral commission & cashback not implemented. F-043 · F-044
- Admin surface ~25 commands vs ~177; no in-bot text customization; pipe-delimited CLI args replace interactive flows.
- `service_time` stores an absolute timestamp in some paths and a day-count in others → on-hold / extend math bugs. F-080 · F-013

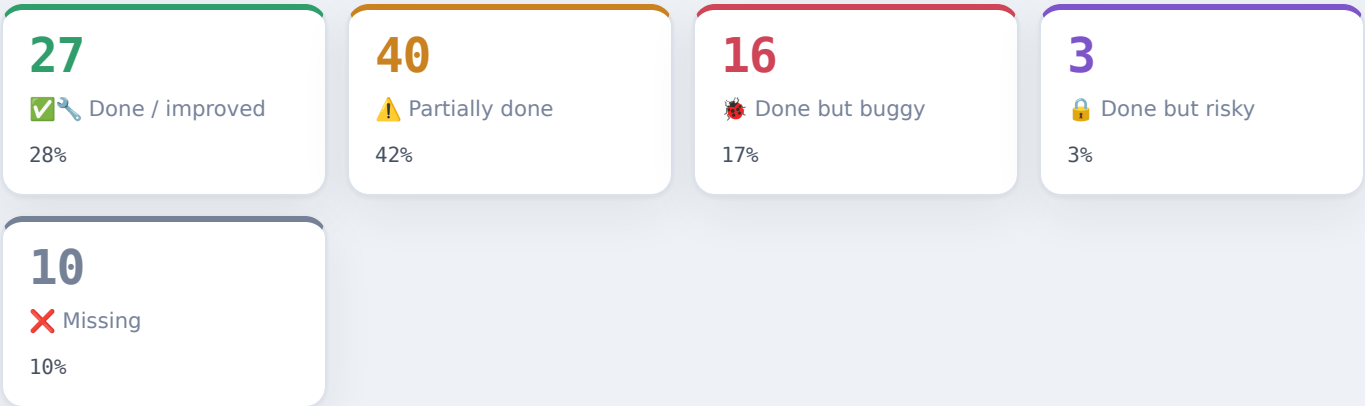
■ Low / hygiene & testing

- **Dead / placeholder code (AI-generation tells):** a shadowed insecure duplicate `miniapp` in `api/__init__.py` (its token action trusts a client `user_id`); `payhooks._report`; `service._expire_from`; the migration `col_defs` loop; `r = web.Application() # placeholder to keep linters calm`; `_TOPUP_UNUSED`; `new_tracking_code`.
- Web panel: no `Secure` cookie flag; inconsistent HTML-escaping; a bcrypt timing-parity "dummy hash" that is invalid and raises. **SQL is parameterized via the ORM — no injection found.**
- Webhook secret and forced-channel checks **fail open** when misconfigured. Weak default `MIRZA_SESSION_SECRET=change-me` doubles as the panel-password encryption key.
- **Testing:** 38 test functions, strong on `initData`, CubePay/Stars math and idempotency — but **no test exercises the settle-topup or bot checkout path**, which is exactly why the suite is green while

the money path is broken. Components are validated in isolation; the end-to-end purchase journey is never assembled.

07 Completion & how it was calculated

Each of 96 parity features is scored by fidelity to baseline (Done/Improved = 1.0, Risky = 0.7, Partial = 0.5, Buggy = 0.2, Missing = 0.0) and averaged. Additions are excluded from the denominator.



READ THE NUMBER CAREFULLY

Completion is concentrated in read/display paths (viewing services, stats, listings, the Mini-App). The transactional core — take payment → provision → notify — is where the buggy cluster sits. A user-journey-weighted score would land well below 55%. Treat 55% as "how much of the surface area is present," not "how ready it is to run a shop."

Weighted completion by functional area				
AREA	COMPLETION		READING	
Web / HTTP surfaces		86%	Solid	
Support & content		75%	Solid	
Service management		63%	Partial — needs work	
Bot UX & onboarding		60%	Partial — needs work	
Payments & wallet		56%	Partial — needs work	
Purchase & provisioning		55%	Partial — needs work	
Data & infrastructure		55%	Partial — needs work	
Scheduled jobs		53%	Partial — needs work	

AREA	COMPLETION	READING
Agent / reseller		50% Partial — needs work
Admin management		41% Weak — major gaps
Growth & marketing		31% Weak — major gaps

08 Recommended next steps

MUST FIX BEFORE REPLACING MAIN — the "make the core actually work" tranche (1-7); land together with an end-to-end test.

- 1 Define `PaymentService.settle_wallet_topup(order, cashback_pct=0)` and cover it with a settlement test — unblocks all gateway top-ups + Stars.
- 2 Set `invoice_id` on gateway product orders in `buy.py::pay_gateway` so purchases provision instead of top up.
- 3 Wire a real bot into `payhooks` settlement — implement buyer DM and the admin report.
- 4 Fix wallet-purchase ordering: charge → provision in one transaction; re-verify balance atomically.
- 5 Fix change-location and manual-sell to carry real volume / expiry.
- 6 Point expiry & volume reminders at the customer; commit `disableconfig`.
- 7 Move ephemeral state to the DB / Redis (aiogram FSM) — removes the race and multi-worker breakage.

REACH REAL PARITY — the second tranche (8-14).

- 8 Restore money-adjacent admin parity: multi-admin gating, per-panel pricing/inbound/test config, product edit/delete, discount %/limits, card-number management, in-bot text customization.
- 9 Rewrite the discount engine (percent + fixed + per-product + usage-limit + expiry) and increment `used_count`.
- 10 Real migrations: replace the `create_all` shim with explicit DDL, and ship the compatibility view the settings redesign depends on.

- 11 Port the lottery job; implement gift drops; connect `/wheelprizes` to the wheel.
- 12 Referral commission & cashback; phone-verification enforcement; the on-hold creation path.
- 13 Harden config: require non-default secrets at boot; separate the encryption key from the session secret; add `Secure` cookies and consistent escaping.
- 14 Add end-to-end tests for the full purchase journey per gateway before re-claiming parity.

09 What could not be verified statically

Deferred to the second (live-run) pass:

- **Panel adapters against real panels.** Only `marzban.py` was deep-read (looks faithful). Marzneshin, Alireza/x-ui, S-UI, Hiddify, WGDashboard (keygen), MikroTik (REST v7), IBSng, Pasarguard, Rebecca and mirza_agent need live create/modify/remove/revoke round-trips.
- **The end-to-end purchase → payment → provision → notify flow** per gateway — confirm the \$6 crashes and the top-up mis-route on a running instance.
- **Alembic** `upgrade` / `downgrade` against real PostgreSQL (JSONB, FKs, the missing compat view, the absent `protocol` table).
- **Gateway callback signatures** against live sandboxes (Zarinpal v4, Aqayepardakht code-70, Plisio/NowPayments webhooks, IranPay poll, CubePay HMAC).
- **The Mini-App inside Telegram** (real `initData`, token exchange, purchase actions), **scheduled jobs** on the real scheduler, **white-label** child-bot dispatch, and the **installer** on a clean host.

10 Coverage statement

Fully read & reasoned about: data model & migration; bot purchase flow; `panels/service.py` ; `payments/service.py` ; payment webhooks; wallet; middleware; webhook entry; admin handlers; all scheduled jobs; web panel; REST + Mini-App API; user misc/menu; `marzban.py` .

Read lightly / trusted pending dynamic verification: individual gateway internals, `extra.py` , `services.py` detail, `whitelabel.py` , `backup.py` , `rates.py` , and 13 of the 14 panel adapters.

This is a complete **breadth-first** pass — every functional area inventoried and mapped, with deep scrutiny on the risk-weighted core. It is *not* a line-by-line audit of

all 89 rewrite and 97 baseline files; the areas above are the known depth boundary, and no partial result is presented as exhaustive.

Mirza Bot migration audit · static analysis pass 1 of 2 · generated 2026-08-25. Baseline origin/main (PHP 8.2 / MySQL) treated as source of truth; rewrite origin/rewrite/python (aiogram / SQLAlchemy / aiohttp / PostgreSQL). Feature IDs F-001...F-096 are stable references into the parity inventory. A companion PMO dashboard summarizes these figures visually.

Remediation brief — `rewrite/python` (for the implementing AI)

To: the assistant that generated the `rewrite/python` branch. **From:** an independent static audit against `main` (the production PHP bot, which is the source of truth). **Purpose:** a precise, ordered work order to reach real parity. Every item below was found by *reading the code*, not running it. Fix in the order given — the CRITICAL block breaks the revenue path and must land first, together with tests.

Ground rules before you start

1. **Do not trust** `PARITY.md`. Several of its  entries are false (called out below). Re-verify each claim against the code.
 2. **Every fix needs a test that would have failed before it.** The current suite is green *because it never exercises the settlement/checkout path* — that is how the CRITICAL bugs shipped. Add end-to-end coverage for the purchase→pay→provision→notify journey per gateway.
 3. **Do not add features while fixing bugs.** Keep additions labelled and separate, as you already did.
 4. File references are `path::symbol` with line hints from the audited commit; re-locate by symbol if lines drift.
-

CRITICAL — online payments are non-functional (fix first, as one tranche)

C1 — `PaymentService.settle_wallet_topup` is called but never defined

- **Where:** called at `src/mirza/api/payhooks.py:76` and `:107`, `src/mirza/jobs/builtin.py:435` (`_settle`), `src/mirza/payments/stars.py:160`. **No definition exists** anywhere (`PaymentService` in `src/mirza/payments/service.py` only defines `settle_direct_buy`).
- **Effect:** every wallet top-up settlement — Zarinpal, Aqayepardakht, IranPay, Plisio, NowPayments, card-to-card, **and Telegram Stars** — raises `AttributeError` at runtime.
- **Fix:** implement `async def settle_wallet_topup(self, order, cashback_pct: int = 0)` on `PaymentService`: credit `order.price` (+ optional cashback) to `order.user_id` via `WalletService.change(..., reason="topup")`, guard against double-credit (the `claim_paid` gate already covers idempotency, but assert it), and return a result the callers can use.
- **Accept when:** a test tops up a wallet through at least one gateway callback and asserts the balance increased exactly once (and once more under a duplicate callback = no double credit).

C2 — Gateway *product* purchases never provision the product

- **Where:** `src/mirza/handlers/user/buy.py::pay_gateway` calls `PaymentService.create_order(str(db_user.id), int(price), gw_name, bot_type="main")` with **no** `invoice_id`. `payhooks.py::settle` only provisions when `order.invoice_id` is set (`if order.invoice_id: settle_direct_buy(...) else: settle_wallet_topup(...)`).

- **Effect:** a customer who selects a product and pays via an online gateway gets a wallet top-up (which then crashes per C1), **not the config they bought**. The product `code` is discarded (used only in the description string).
- **Fix:** mirror what the Mini-App already does correctly (`src/mirza/api/miniapp.py::mini_purchase`): create the `Invoice` row (status `pending`) *before* the order, and pass `invoice_id=<that token>` into `create_order` . Then `settle_direct_buy` will fire on callback.
- **Accept when:** paying a product order via a gateway callback creates/activates exactly one config for the buyer and no phantom wallet credit.

C3 — Buyer is never notified and the admin report is a no-op

- **Where:** `src/mirza/api/payhooks.py` — `settle / settle_tuple` call `_notify_user(None, order.user_id, text)` (bot hard-coded to `None` , so `_notify_user` returns immediately). `_report()` (`payhooks.py:41`) builds `chat` then does `_ = chat, kind, text # delivery via scheduler ctx in production run` and sends nothing.
- **Effect:** successful payments are silent to the customer, and no payment report reaches the admin channel/topic (a rich feature in `main`).
- **Fix:** thread a real `Bot` instance into the settlement path (via the aiohttp app context / job context) and actually send the buyer DM; implement `_report()` to post to the configured report channel/topic (resolve `topicid` for `paymentreport`). Do not swallow the send silently — log failures.
- **Accept when:** a settled payment produces a buyer DM and an admin-channel message in an integration test with a fake bot.

C4 — Wallet purchase provisions and commits *before* charging

- **Where:** `src/mirza/handlers/user/buy.py::pay_wallet` . Order of operations: `panels.create_user(...)` (which **commits** the invoice inside `panels/service.py::create_user` , line ~126) → then `WalletService.change(-price)` . On failure it calls `session.rollback()` , but the invoice is already committed and the config already exists on the remote panel.
- **Effect:** a failed/insufficient charge yields a **free provisioned config**. Because balance is checked earlier in `buy_confirm` and deducted later, two concurrent purchases can both pass the check and both provision → double-spend.
- **Fix:** charge the wallet atomically *first* (conditional decrement: `UPDATE user SET balance = balance - :price WHERE id=:id AND balance >= :price` returning affected rows), and only provision if the charge succeeded; on provision failure, refund in the same transaction. Never commit the invoice before the charge is secured.
- **Accept when:** a concurrency test with balance = one purchase's price and two simultaneous buys results in exactly one config + one debit.

● HIGH — regressions and broken behaviour

H1 — `change_location` creates a dead, empty config

- **Where:** `src/mirza/panels/service.py::change_location` → `create_user(new_panel, product_code, username, data_limit=0, expire_ts=_now())` .

- **Effect:** the migrated config has **zero volume and expires immediately**; the user's remaining quota/time is dropped. `main`'s `changeloc` preserves remaining volume and time.
- **Fix:** read the current config from the old panel (`data_user`), compute remaining volume (`data_limit - used_traffic`) and remaining time (`expire_at - now`), create on the new panel with those, then remove from the old.

H2 — `manual_sell` provisions zero volume and ignores the product

- **Where:** `src/mirza/handlers/admin/__init__.py::manual_sell` → `create_user(..., data_limit=0, expire_ts=now+30d)`, and `_default_panel_name` picks the first panel arbitrarily.
- **Fix:** look up the product by `code`, use its `volume_gb` / `service_days` / `location`; fail clearly if the product doesn't exist.

H3 — Expiry & volume reminders go to the admin channel, not the customer

- **Where:** `src/mirza/jobs/builtin.py` — `NotificationsJob` and `VolumeWarnJob` only call `ctx.send_report("notifications", ...)`. `main`'s `NoticationsService.php` DMs the *customer* with renew buttons.
- **Effect:** the renewal-reminder engine never reaches buyers — a direct revenue regression.
- **Fix:** DM `inv.user_id` with a renew CTA; optionally also mirror to the report channel. Respect the per-invoice `notifications` flags (already present) to avoid repeats.

H4 — `disableconfig` never persists its status change

- **Where:** `src/mirza/jobs/builtin.py::DisableConfigJob` disables on the panel and sets `inv.status = "disabled"` but has **no** `await s.commit()` (unlike sibling jobs).
- **Effect:** status change is rolled back on session close; the job re-disables the same invoices every tick and the DB never reflects reality.
- **Fix:** commit at the end of the sweep.

H5 — Multiple admins unimplemented (contradicts `PARITY.md` "✅")

- **Where:** `src/mirza/handlers/admin/__init__.py::AdminGate` ("only primary admin passes") and `manage.py::AdminGate` gate solely on `settings.admin_number`. The `Admin` table and `rule` column are never consulted for bot gating.
- **Fix:** gate on membership in the `admin` table (with `rule` /level checks where `main` differentiates), not a single env id. Keep the primary admin as a superuser.

H6 — `uptime_node` authenticates with the *encrypted* password

- **Where:** `src/mirza/jobs/builtin.py::UptimeNodeJob` builds `MarzbanAdapter({... "password_panel": panel.password_panel_encrypted})` — the Fernet ciphertext, not the plaintext.
- **Fix:** decrypt via the same path `PanelService._adapter_for` uses (`_decrypt_secret`), or route through `PanelService` instead of constructing the adapter directly.

H7 — Migrations are `create_all` in disguise, with dead code

- **Where:** `alembic/versions/0001_initial.py::upgrade` loops `Base.metadata.sorted_tables` and calls `table.create()` ; lines ~30–42 build `col_defs` via `col.compile(...)` and **never use them**. No real DDL is captured, and the compatibility view the `Setting` model docstring promises (for the legacy flat `setting` columns) does not exist.
- **Fix:** generate a real autogenerated migration with explicit `op.create_table(...)` per table; add the `setting` compatibility view (or a data migration) if you keep the KV redesign; delete the dead `col_defs` loop.

H8 — Process-local state breaks restarts and multi-worker

- **Where:** module-level dicts: `buy._pending` , `misc._TOPUP` , `admin.__init__._BROADCAST` , `routes/webpanel._SESSIONS` .
- **Effect:** all in-flight purchase/top-up/broadcast/admin-session state is lost on restart and is not shared across workers; also enables the C4 race.
- **Fix:** use aiogram FSM with a persistent (Redis) storage backend, and persist web sessions in the DB or a shared store. `main` persisted step state in the DB (`user.step` , `Processing_value*`).

🟡 MEDIUM — incomplete / disconnected features

M1 — Discount engine ignores most of the model

- **Where:** `src/mirza/handlers/user/buy.py::discount_code` applies only `price_discount` (fixed). `discount_percent` is ignored, `used_count` is never incremented (so `usage_limit` is unenforceable), `expires_at` is unchecked, and `DiscountSell` (per-product codes) is unused.
- **Fix:** apply percent *and* fixed, enforce `usage_limit` by incrementing `used_count` transactionally at redemption, honour `expires_at` , and support `DiscountSell` .

M2 — Gift codes are inert and the consumed-count is wrong

- **Where:** `manage.py::add_giftcode` creates a `Discount` with `discount_percent=0` / `price_discount=0` → no actual discount. `manage.py::list_giftcodes` builds `{g.code: g for g in ...}` (keyed by code), so `used` collapses to ≤ 1 entry per code and the consumed count is always 0/1.
- **Fix:** let `/giftcode` set a real value; count consumption with `COUNT(*) GROUP BY code` over `Giftcodeconsumed` .

M3 — Lucky-wheel and wheel-prize config are disconnected

- **Where:** `misc.py::lucky_wheel` uses a hard-coded `WHEEL_PRIZES` ; `manage.py::set_wheel_prizes` writes the `wheel_prizes` setting that the wheel never reads. Legacy wheels also support weighted probabilities.
- **Fix:** read `wheel_prizes` from settings; support weights if `main` does.

M4 — Scheduled lottery missing; gift job stubbed

- **Where:** `main` has `cronbot/lottery.php` (periodic draw) — **no equivalent job** in `jobs/builtin.py` (header even says "16 jobs"; `main` has 17). `GiftJob.run` is `return None`.
- **Fix:** port the lottery draw; implement scheduled gift drops per `gift.php`.

M5 — Referral commission & cashback not implemented

- **Where:** `menu.py` captures `referrer_id` and `misc.py::affiliates` shows a count, but nothing pays commission on a referred purchase. `Affiliate.porsant_one_buy`, `reagent_report`, `status_commission` are unused; the only "cashback" is a hard-coded `cashback = 0` in `jobs/builtin.py::_settle`.
- **Fix:** on a referred user's first/every purchase (per `main`'s rule), credit the referrer's wallet and record `reagent_report`; implement cashback where `main` grants it.

M6 — Free trial ignores per-panel config; weak gating

- **Where:** `misc.py::free_trial` hard-codes 1 GB / 3 days and passes `product_code=""`; it ignores the panel's `time_usertest` / `val_usertest`, and gates only on "already has a test invoice" (no phone/channel/limit checks; `on_hold_test` unused).
- **Fix:** read per-panel trial volume/time; apply the same gates `main` enforces.

M7 — On-hold path is dead; `service_time` semantics are inconsistent

- **Where:** `OnHoldJob` processes `status == "on_hold"`, but **no handler ever sets `on_hold`**. Also `create_user` writes an **absolute expiry timestamp** into `invoice.service_time`, while `OnHoldJob` (`_days_s`) and `service._expire_from` treat `service_time` as a **day count** → nonsensical expiry math.
- **Fix:** pick one representation for `service_time` and use it everywhere; wire the on-hold creation path so the job has something to activate.

M8 — Forced-channel join is single-channel and fails open

- **Where:** `middleware.py` enforces only a single `channel_lock` setting; the multi-row `channels` table (populated by `/addchannel`) is never enforced. `_check_member` does `channel.lstrip("@")` (likely mis-resolves the chat) and returns `True` on exception (fail-open).
- **Fix:** enforce all rows in `channels`; pass a valid chat identifier to `get_chat_member`; decide fail-open vs fail-closed deliberately (legacy fail-open is defensible but note it).

M9 — Admin surface is ~25 commands vs ~177 in `main`

- Missing/critically-thin admin capability (implement to reach parity): **in-bot text/message customization** (edit any bot string — a headline `main` feature, entirely absent); **per-panel pricing/inbound/test-account/`method_username`/extend editing**; **product edit/delete**; **card-number management**; **report-topic configuration**; user limits (`maxbuyagent` / `changeloclimit` / `expireset` / verify); broadcast with media/pin/forward (currently text only, and it opens one DB session per user — batch it); agent types/pricing (`n` / `n2`). Also add multiple username-generation methods (only `random` / `number` exist).

M10 — Mini-App wallet purchase never completes

- **Where:** `api/miniapp.py::mini_purchase` with `method="wallet"` creates a pending order but nothing deducts the wallet or settles it (no callback for the wallet method).
 - **Fix:** settle wallet-method purchases inline (charge + `settle_direct_buy`) instead of leaving a dangling pending order.
-

● LOW — hygiene, dead code, config safety

- **L1 — Remove dead/placeholder code (AI-gen tells):** the shadowed **insecure** duplicate `miniapp` in `api/__init__.py` (lines ~73-124; its `token` action trusts a client `user_id` — dangerous if ever re-wired); `payhooks._report` no-op; `service._expire_from` (unused); the migration `col_defs` loop; `api/__init__.py:320` `r = web.Application() # placeholder to keep linters calm`; `admin.__init__._TOPUP_UNUSED` ; `manage.new_tracking_code` .
 - **L2 — `protocol` table missing** from the model (present in `main`). Add it if protocol-level config is used.
 - **L3 — Web panel is list-only** (`routes/webpanel.py`): no product/panel/user/discount editing. Either build the CRUD or make the bot the source of truth — but something must be able to edit these.
 - **L4 — Config safety:** `MIRZA_SESSION_SECRET` default is `change-me` and it doubles as the Fernet key for panel passwords → creds encrypted under a known key. Refuse to boot on the default; use a **separate** key for panel encryption vs. web sessions. The webhook-secret check and channel check both fail open when unset — require them explicitly.
 - **L5 — Web panel security nits:** set the session cookie `Secure` ; escape all columns in `_table` (currently inconsistent); the bcrypt timing-parity "dummy hash" is not a valid bcrypt string and raises (defeating the intent). SQL is parameterized via the ORM — no injection found, keep it that way.
-

Suggested sequencing

1. **C1-C4 together, with an end-to-end money-path test** (this is the whole point of the product).
 2. **H1-H8** (regressions + infra).
 3. **M1-M10** (feature completeness).
 4. **L1-L5** (hygiene) — L1 and L4 are quick and worth doing alongside C/H.
-

What was NOT statically checkable (verify while you fix)

Panel adapters against real panels (only `marzban.py` was read in depth — it looks faithful; Marzneshin, Alireza/x-ui, S-UI, Hiddify, WGDashboard keygen, MikroTik REST v7, IBSng, Pasarguard, Rebecca, mirza_agent need live round-trips); Alembic up/down on real PostgreSQL; each gateway's verify logic against live sandboxes; the installer; white-label child-bot dispatch.

ox-alpha's fixes, independently verified

Every fix the implementer claimed was re-read in the actual code of `rewrite/python@c7e0fee`, and the new money-path tests were read line-by-line — the same "trust nothing until read" discipline applied to `PARITY.md` in pass 1.

RE-AUDITED HEAD
c7e0fee

FIX COMMIT
74dba0e

FILES CHANGED
14

NEW MONEY TESTS
3

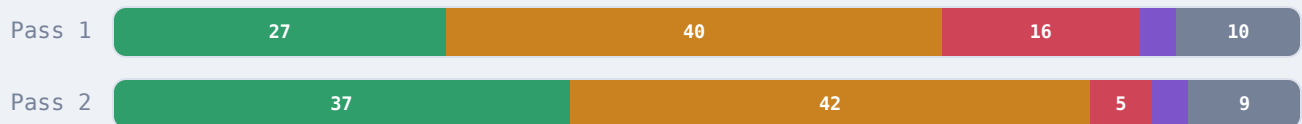
METHOD
static

✓ **Accept tranche 1 — real, tested, safe to build on**

55% → **64%**
WEIGHTED PARITY

The CRITICAL money-path block went from **0 of 4 working** to **≈3.5 of 4**: a customer can now pay via an online gateway and actually receive their config, and a wallet top-up credits instead of crashing. The response doc is candid and accurate. Two residuals remain (§3) and the branch is not yet production-ready — but this is a materially different, much healthier branch than pass 1.

01 Status movement



Done 27→**37** Partial 40→**42** Buggy 16→**5** Risky 3→**3** Missing 10→**9**

Eleven items left the buggy column: F-006/022/030/032/033 (the whole gateway/Stars settlement chain), F-016 (change-location), F-048/075 (lucky wheel + prize config), F-078 (disable-config commit), F-082 (node uptime). F-005 moved buggy → risky (ordering fixed, race open) and F-068 missing → partial (admin gating, level 1).

02 Independent verification

CRITICAL — money path

ID	CLAIM	VERDICT	EVIDENCE
----	-------	---------	----------

ID	CLAIM	VERDICT	EVIDENCE
C1	settle_wallet_topup restored	✓ confirmed + test	Defined <code>service.py:135</code> ; test asserts exactly-once credit + idempotent replay.
C2	gateway purchase sets invoice_id	✓ confirmed + test	<code>pay_gateway</code> writes pending invoice first; test drives real <code>settle()</code> , asserts invoice→enable, ledger empty .
C3	buyer DM + admin report	🕒 partial	Bot threaded for 5 gateways. Residual: cubepay's <code>settle_tuple</code> still notifies <code>None</code> & skips <code>_report</code> . Topic threading deferred.
C4	charge-first + atomic decrement	🕒 partial	Ordering fixed (charge→provision→refund). Residual: <code>change()</code> is read-modify-write, not atomic — concurrency double-spend still possible on PostgreSQL.

HIGH

ID	CLAIM	VERDICT	EVIDENCE
H1	changeloc preserves quota	✓ confirmed	Reads source config, carries remaining bytes/days, removes old.
H4	disableconfig commits	✓ confirmed	<code>await s.commit()</code> added at <code>builtin.py:88</code> .
H5	DB-backed admin gating (L1)	✓ confirmed	Reads <code>admin</code> table (60s cache); rule levels deferred (H5b).
H6	uptime_node decrypts	✓ confirmed	Decrypts via <code>_decrypt_secret</code> before building the adapter.
H2 · H3 · H7 · H8	deferred to tranche 2	— accurate	Not claimed fixed; pass-1 findings stand (manual_sell, reminder DMs, real migration, persistent state).

MEDIUM / LOW

ID	CLAIM	VERDICT	EVIDENCE
M2	gift code value + count	✓ confirmed	Real percent; count via <code>COUNT(*) GROUP BY</code> . Redemption increment with M1.
M3	wheel reads config	✓ confirmed	<code>_wheel_prizes</code> reads the setting, falls back to defaults.
M10	mini-app wallet inline settle	✓ confirmed	charge→provision→refund (402/502).
L1	dead/insecure code removed	✓ confirmed	Shadowed insecure <code>miniapp</code> , placeholder app, <code>_TOPUP_UNUSED</code> , <code>_expire_from</code> all gone.

ID	CLAIM	VERDICT	EVIDENCE
L4	boot refuses default secret	✓ confirmed	<code>server.py:38-41</code> SystemExit on empty token / default session secret.
M1 · M4-M9 · L2 · L3 · L5	deferred	— accurate	Honestly labelled; matches pass 1.

03 Residuals the response under-states

ACT ON THESE — THEY FINISH THE CRITICAL BLOCK PROPERLY

1. **C4 is not race-safe.** The provision-before-charge bug is genuinely fixed, but "atomic conditional decrement" is inaccurate — `WalletService.change` reads then writes in Python with no row lock. Two concurrent wallet buys can still double-spend on PostgreSQL. **Fix:** a single atomic `UPDATE "user" SET balance = balance - :amt WHERE id=:id AND balance >= :amt` (act on rowcount), or `SELECT ... FOR UPDATE` before the check.
2. **cubepay settlement is still silent.** `settle_tuple` (cubepay only) passes `bot=None` and skips `_report`. **Fix:** thread the bot into `settle_tuple` like `settle` does, and call `_report` there.

Test-suite check. The response reports "45 passed." I did not execute the suite (still a static pass) but read `tests/test_money_path.py` in full: the gateway-provision and top-up tests are **genuine** — they drive the real `settle()` and assert the criteria that matter (invoice activated, order paid, wallet untouched / credited once), and would fail on the pass-1 code. The third test only unit-checks the charge refusal — it does not exercise the full flow or the concurrency case. Net: honest tests, not gamed; the one missing dimension is exactly the C4 residual above.

04 What still stands between this and replacing main

- 1 **Finish CRITICAL:** close the C4 atomic-debit race and the cubepay notify/report gap — both small.
- 2 **Tranche-2 revenue items** (implementer's own list, agreed): customer expiry-reminder DMs, discount engine + redemption increments, referral commission & cashback, trial per-panel config, on-hold + `service_time` semantics, real DDL migration + settings view, persistent FSM/sessions.
- 3 **Tranche-3 breadth:** in-bot text customization and the rest of the admin surface, lottery/gift jobs, web-CRUD decision.

Dynamic pass (still owed): execute the 45 tests; each panel adapter against a real panel;

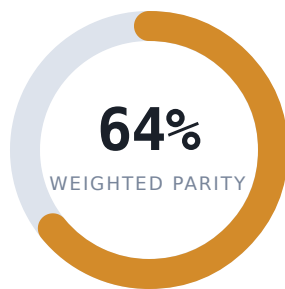
- 4 Alembic up/down on PostgreSQL; gateway callbacks against sandboxes; installer; white-label dispatch.

Re-audit pass 2 · verified statically against `rewrite/python@c7e0fee` · 2026-08-25. Every ✓ was re-read in the source; the two residuals in §3 were found by reading past the response's summary, not taken from it. Companion to the pass-1 report and PMO dashboard.

Mirza Bot — PHP → Python Parity

Static-analysis audit of the rewrite/python branch against main (production PHP). Baseline = source of truth.

```
RE-AUDIT post tranche-1 · static
rewrite head c7e0fee
parity 55% → 64%
features scored 96
2026-08-25
```



After tranche-1, the money path works: a customer can pay via a gateway and receive their config, and wallet top-ups credit instead of crashing.

Parity moved **55% → 64%**; the critical block went from **0/4** to **≈3.5/4**. Verified by re-reading every fix in code. Two residuals remain and the revenue-retention features (customer reminders, discounts, referrals) are still tranche-2.

✓ Tranche-1 accepted · not yet production

37

✓ Done / improved

42

⚠ Partial

5

🐛 Wrong / buggy

3

🔒 Risky

9

✗ Missing

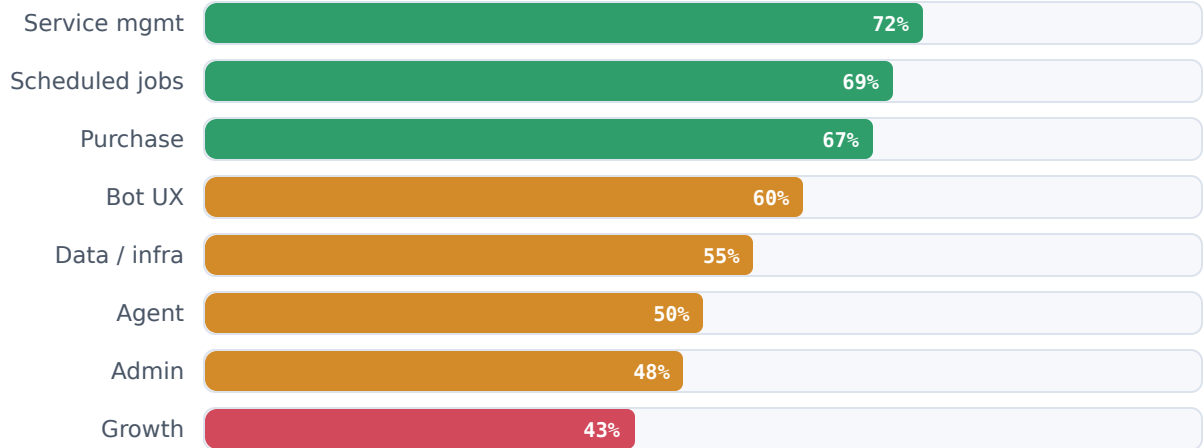
STATUS BREAKDOWN



✓ Done / improved	37	39%
⚠ Partially done	42	44%
🐛 Done but buggy	5	5%
🔒 Done but risky	3	3%
✗ Missing	9	9%

PARITY BY FUNCTIONAL AREA





TOP LAUNCH-BLOCKING RISKS

HIGH

Wallet debit is not race-safe

Charge-before-provision is fixed, but WalletService.change is read-modify-write (no row lock / atomic UPDATE) — two concurrent wallet buys can still double-spend on PostgreSQL.

C4 · F-005

HIGH

Expiry reminders miss the customer

Renewal/volume warnings still post to an admin channel, not the buyer — the renewal engine never reaches paying users (tranche-2).

F-076 · F-077

HIGH

Migrations still a create_all shim

Initial migration loops model metadata instead of real DDL; no compatibility view for the redesigned settings table (tranche-2).

F-094

HIGH

Admin surface still a fraction

~25 commands vs ~177; in-bot text customization and per-panel/product editing still absent — the largest remaining block.

M9

RESOLVED

Gateway settlement (was critical)

RESOLVED in tranche-1: settle_wallet_topup restored, gateway purchases set invoice_id, buyer/admin notified for 5 gateways (cubepay notify still open).

C1 · C2 · C3

AREA DETAIL — STATUS MIX

Purchase & provisioning

67%

Service management

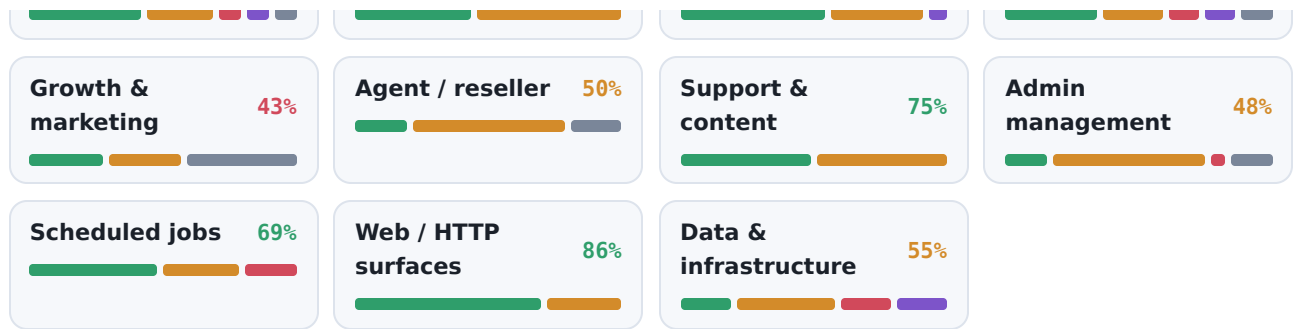
72%

Payments & wallet

82%

Bot UX & onboarding

60%



How completion is calculated. Each of 96 parity features is scored by fidelity to baseline — Done/Improved = 1.0, Risky = 0.7, Partial = 0.5, Buggy = 0.2, Missing = 0.0 — and averaged. Additions in the rewrite (ledger, idempotency, webhook-secret, encryption, plugin registry) are excluded from the denominator. The score measures how much surface area is present, **not** operational readiness of the revenue path.

Re-audit (post tranche-1): the three critical payment defects are fixed and now covered by genuine money-path tests — verified by re-reading every change in `rewrite/python@c7e0fee`. Two residuals remain: the wallet debit is not race-safe (`change()` is read-modify-write, not an atomic conditional decrement), and cubepay's `settle_tuple` path still doesn't notify the buyer or report. · Scope: static pass only — no code executed. Executing the 45-test suite, panel-adaptor round-trips and end-to-end gateway flows remain for the dynamic pass.